TimeWeave: Day 1 Progress

Clash-free college timetables, generated in seconds

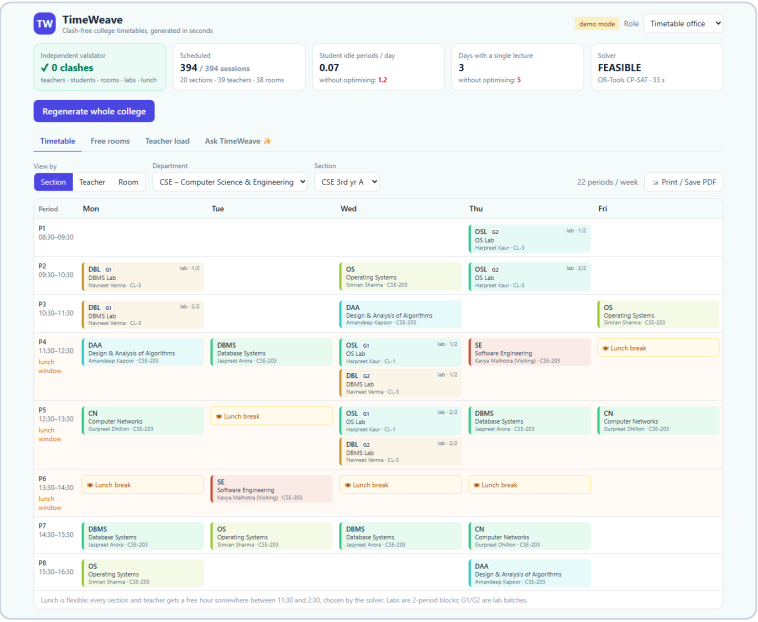
Dilnaz Grewal
Hackathon Day 1 · Wed 21 Oct 2026
github.com/dilnazgrewal/Timetable

Problem: college timetables are built by hand over weeks. They still clash, leave students waiting hours between lectures, and break whenever one class moves. **Built today:** a working prototype. It generates the whole college's timetable with Google OR-Tools **CP-SAT**, proves it clash-free with an **independent validator**, and shows it on desktop and phone, with a free-room finder, dashboards, an AI assistant and PDF printouts.

✓ 0 clashes (independent validator)	**394/394** sessions placed · 20 sections, 39 teachers, 38 rooms	**~33 s** to generate the whole college	**1.2→0.07** student idle periods/day (unoptimised → optimised)	**5→3** days with a single lecture	**12/12** automated tests passing

Features completed today (v0.1)

- ✅ **Realistic college data:** CSE, IT, ECE, ME, CE and Applied Sciences; G1/G2 lab batches; shared computer labs; teachers shared across departments; a visiting teacher available Tue/Thu only.
- ✅ **One-click whole-college generation** (CP-SAT) with hard rules that are never broken: no teacher, student or room double-booking; availability; labs as 2-hour blocks in the right lab; daily limits.
- ✅ **Flexible lunch:** every section and teacher gets a free hour somewhere in 11:30–2:30, chosen by the solver.
- ✅ **Student-friendly days:** the solver minimises idle gaps, single-lecture days, teacher gaps and long streaks.
- ✅ **Pre-checks** explain impossible inputs in plain English before solving.
- ✅ **Independent validator** re-checks every timetable and shows "✓ 0 clashes".
- ✅ **Timetable grids** by section, teacher or room, with a one-day-at-a-time view on phones.
- ✅ **Free-room finder** with a "Right now" button and how long each room stays free.
- ✅ **Dashboards:** teaching load per teacher; classroom and lab utilisation.
- ✅ **AI assistant "Ask TimeWeave"** (Claude): answers from live data with 4 read-only tools and cannot change anything. Tested with the live API.
- ✅ **Print / Save as PDF:** one A4 page per section, teacher or room.
- 🟡 **Roles** (office, coordinator, teacher, student) as a demo switcher; real login is planned for Day 3.
- ✅ **Docs and history:** CLAUDE.md, PLAN.md, DESIGN.md, README; 13 git commits.



The generated timetable for CSE 3rd yr A: validator badge, metrics, labs for G1/G2, and each day's 🍽 lunch break.

Architecture

Input	Seed data now → Excel/CSV upload (Day 2)
Pre-checks	"Prof. X has 32 h but is free only 28 h"
CP-SAT solver	Hard rules + minimise gaps, single-lecture days and long streaks
Validator	Separate code re-checks everything → "✓ 0 clashes"
Saved timetable	Weekly base (+ daily overrides from Day 3)
Outputs	Grids · free rooms · dashboards · AI assistant · PDF

Stack: Python, FastAPI, Google OR-Tools CP-SAT, Tailwind + Alpine.js (one page, phone-first), Claude API, pytest, git.

Testing: proof of no clashes (12/12 pass)

- The generated timetable validates clean, and every session is placed exactly once.
- Deliberately injected errors are all caught: student clash, room clash, wrong room type, unavailable teacher, missing lunch, lab running past the end of the day, missing session, unknown data.
- An overloaded teacher is explained by the pre-check, and everyone gets a lunch hour.
- Manual checks: the PDF fits one page, the free-room finder matches the grids, and the AI assistant's answer matches the grid.

Next steps to submission

Day 2 real data	Excel/CSV upload with row-level errors · check an existing hand-made timetable · drag-and-drop editor with live clash warnings · versions, rollback and audit log · Excel export
Day 3 daily operations	Teacher leave → ranked substitutes → coordinator confirms · student "Today" view · free rooms that account for bookings · Google login with roles · calendar feed
Day 4 submission	Minimal-change repair with a preview · chatbot proposes, a person confirms · "why no timetable" explanations · room QR pages · deploy

Deliberate Day 1 shortcuts

- Data lives in memory plus a JSON file; a database comes on Day 2.
- Roles are a demo switcher; real login comes on Day 3.

Run: `pip install -r requirements.txt` → `uvicorn app.main:app --port 8000` → open `localhost:8000`

Full plan: PLAN.md